

Distributed Computing Using Sentiment Analysis Through Different Models

Anastasios (Tasos) Lilis

California State University, Long Beach

Department of Computer Engineering & Computer Science

August 10th 2025

Abstract

This summer, I have chosen to develop a project that implements a distributed sentiment analysis system that compares six different approaches to sentiment text analysis using the Ray distributed framework. For my data set, I have chosen reviews from Disneyland Hong Kong to be processed through these different models. These models include; an LLM-based (DialoGPT-small), API-based (simulated), 3-class Hugging Face (RoBERTa), manual rule-based analysis, Hugging Face (DistilBERT), and VADER (Valence Aware Dictionary and sEntiment Reasoner). I have chosen these models as they all provide something different with each one allowing me to track their performance metrics like execution time, memory usage, and model agreement. These models will be measured and visualized using Python and related libraries to help us understand the differences in each model. This program has a distributed architecture that enables parallel processing, trade-offs, and different approaches to a similar outcome. While there is some discourse when it comes to what each model thinks, this offers insights as to the overall effectiveness of different sentiment analysis methods in this distributed computing environment.

Introduction

Since the wide scale availability of the internet has grown, people have had the opportunity to voice their opinion on just about anything. This is both great and overwhelming, as so many reviews or opinions can be hard to digest for customers, companies, and critics alike. My project aims to mitigate this problem by using sentiment analysis through different models on a distributed computing architecture. Sentiment analysis is the process of identifying and categorizing opinions from a piece of text using computational techniques, in order to determine the author's attitude towards a topic, product, or place [1 2 3]. In the case of this project, we will be determining if the author feels positive, negative, or neutral about their experience at Disneyland Hong Kong. For this report, we will be analyzing one thousand reviews over six different models. This means that distributed computing is necessary for the scale, speed, and resource utilization we need to accomplish. We will also be using parallel processing to have multiple reviews being analyzed at once. Using the Ray framework in Python allows us to use a simple API call to convert exciting functions to distributed tasks, automatic resource management, fault recovery, and cross-platform accessibility. Ray is an open-source unified framework for scaling AI and Python applications like machine learning. It provides the compute layer for parallel processing so that you don't need to be a distributed systems expert. [4] The six different models will be using include; DiabloGPT-Small (a LLM-based approach that can help generate sentiment classifications through text generation), API-based (a simulation of cloud-based sentiment analysis API), 3-class Hugging Face (RoBERTa) (a fine tuned model meant specifically for sentiment analysis, with three true outcomes), Manual (simple rule based approach that matches keywords by assigning a value to them), Hugging Face (DistilBERT) (a distilled version of bert model used for sentiment analysis), and VADER (Valence Aware

Dictionary and sEntiment Reasoner) (used specifically for lexicon based approaches for small reviews). The point in having all these models is not necessarily to improve accuracy across the system, but rather understand how each model reacts to different reviews and see the differences and trade-offs between them. The main differences we are focused on are the speed, accuracy, and resource usage, while also trying to find similarities between the models. While this project is small in comparison to what could realistically be accomplished, it's quite interesting to see the outputs a small system can produce.

Design / Methods

As mentioned in the introduction, this program's system architecture uses a distributed system with the Ray framework in Python. Using the Ray framework was crucial in processing large volumes of text in an efficient manner. The reason I choose this framework over other ones, is because it is a newer design and simplicity in converting Python functions into distributed tasks. This also allows for tasks to be sent across multiple worker nodes, enabling parallel processing for each review. Each of our models is implemented as a Ray remote function. Ray automatically handles the task distribution, result collection, and resource management. This is important for us to monitor the memory usage before and after each execution of the model.

To run this system we need the following programming components, *main.py* which is the main control center for this program. It is responsible for loading in the data, distributing the tasks, and aggregating the results. It takes in CSV files and processes them through the pandas library, to extract only the written reviews. These reviews are then distributed among different workers using the ray library's functions. Each processing stage is monitored and recorded for their memory usage. The results are then sent to a structured JSON file for visual analysis. The next file we use is *tasks.py*, this engine contains our six different sentiment analysis model

implementations for our distributed execution. We make sure that; our heavier models are loaded once per worker so that we avoid any repeated initialization, each function has garbage collection to prevent any memory leaks during our distributed processes, and we also use try catch blocks to ensure any failures don't lead to the system crashing. Lastly we have the *visualize.py*, This is where we transform all of the raw data collected from our other two programs and process them through some insightful comprehensive visuals using the Matplotlib to create graphs. Some of these graphs include execution time, memory usage, and agreements among the six different models. With these three systems, we are able to create a functional sentiment analysis program that allows for us to examine how each system reacts to different reviews.

Six different sentiment analysis approaches / models were selected to be used for a comprehensive coverage of the sentiment analysis idea. They range from simple rule based approaches, to large language models. This allows us to see the trade offs from accuracy and speed. To start, we have our rule based or “manual” model. This model is the simplest using keyword matching from a predefined list of what is considered to be positive or negative. This offers the fastest approach but it is the least sophisticated as it only focuses on key terms rather than tone or language. Next we use a simulated API approach that mimics commercial cloud use for sentiment analysis providing a fast and suitable response. We then move on to 3 Class Hugging Face (RoBERTa), a model that is specifically tuned and optimized for three class sentiment analysis (positive, negative, and neutral). A solid balance of accuracy and efficiency. Next we have Hugging Face (DistilBERT), another version of BERT that is optimized for sentiment analysis while also being accurate and not too heavy on memory. Valence Aware Dictionary and sEntiment Reasoner or VADER for short, is a lexicon based approach design for

informal text (usually through social media). Last but not least we have DiabloGPT-Small, which is a smaller large language model (LLM) compared to some of the bigger names. Like other LLMs, it takes text generation providing a sophisticated analysis on each segment, but it costs the most time and computational power. I considered using larger models like; ChatGPT, Llama 2, and Mistral. After further research and attempts, the models either are inaccessible with a paid account needed for a certain amount of request, or the models require too much memory to be processed leading to my computer crashing. For the sake of this project I have decided to stick with a group of smaller models than a few larger models.

Lastly we have the data set we are working with. The dataset comes from a very large list of Disneyland reviews from all of their parks. For this project, we will be looking at the first 1000 reviews from the CSV file to analyze as they all come from Disneyland Hong Kong. This seemed to be the sweet spot for this project providing enough information to compare each model's performance and results, while also not overloading the system as my computer starts to heat up but not shut down.

Implementation

Briefly mentioned in the design section, the system architecture here is built between three different python programs using a variety of libraries. For this section we will be breaking down each program and explaining their implementation. We have our main program *main.py* that acts as the orchestrator for the system. It first starts with the system initializing the Ray distributed computing framework, which allows us to have parallel processing. We then load the data in from our CSV file, only processing the first 1000 reviews. We then make sure we are monitoring the memory for later evaluation. This is done through the distributed processing pipeline. We then move on the execution of each sentiment analysis model, we aggregate the

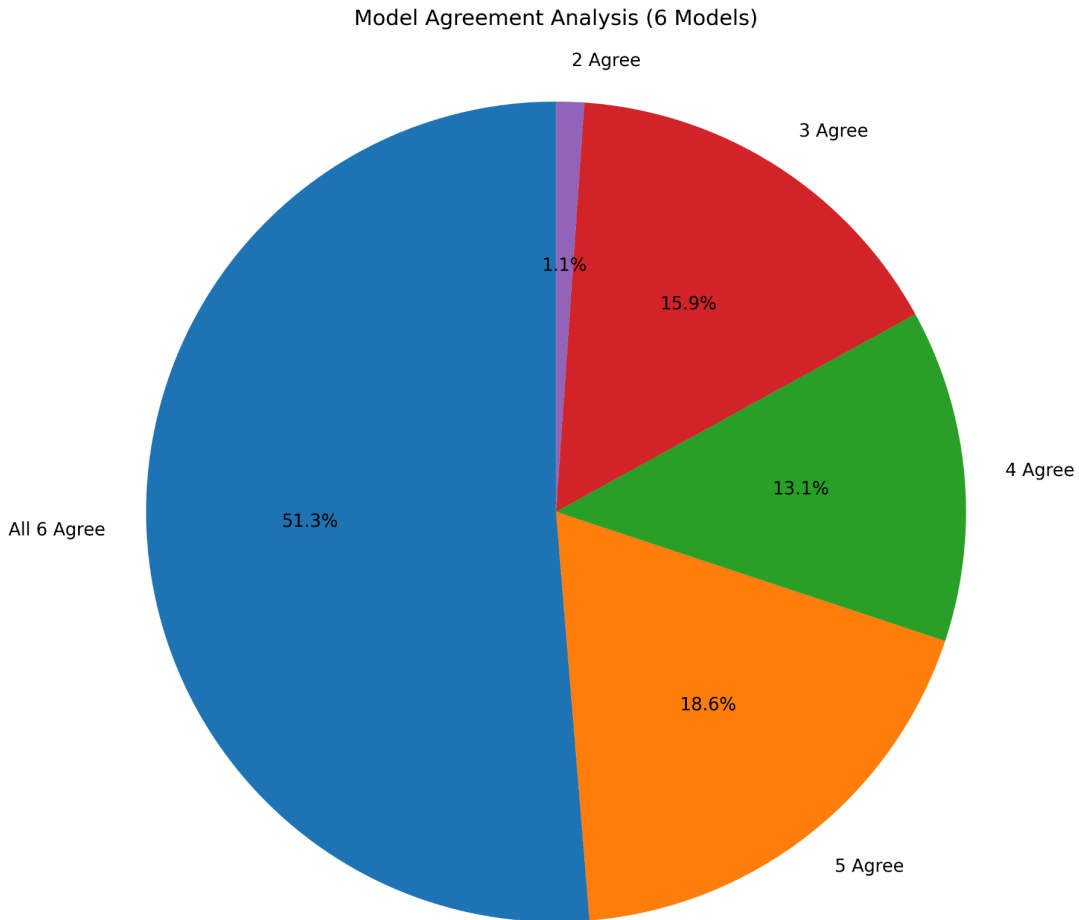
results from the distributed workers into different JSON files for further analysis. Throughout the program we make sure to save any important data used in our graphs.

Next we have our *tasks.py* program, this is the engine that contains the six sentimental analysis implementations. Each of these are designed for distributed execution through Ray's remote function system. We first start with some global variables to determine which words are considered to be positive and which words are negative. This also helps us ensure a consistent resource sharing across the ray workers. We then create pipeline analysis for each and every model we have used.

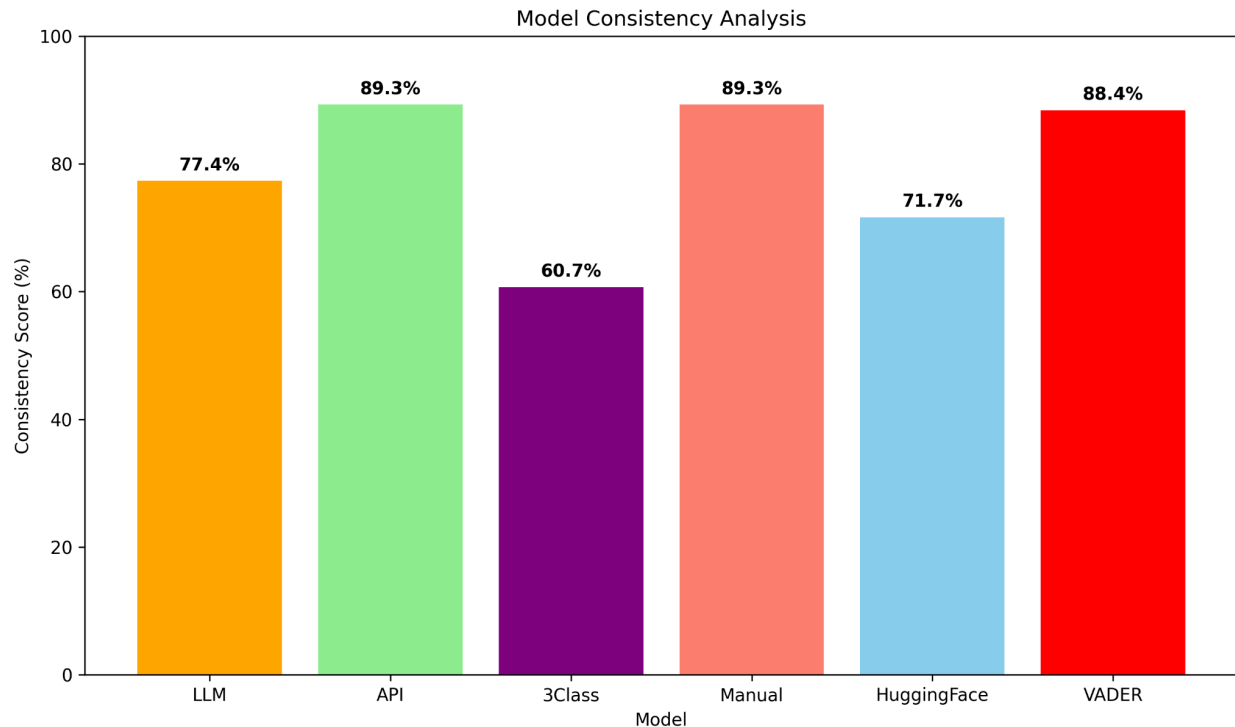
Lastly we have our *visualize.py* program. This engine processes the results from the distributed pipeline to generate comprehensive performance analysis. We first load our data from our saved JSON file and analyze the data. We then create a comprehensive performance tracking across all models for subjects like; execution time, memory usage, and model agreement rates. We also track subjects text length analysis, model consistency, overall positive and negative reviews. Explanation of results will be in the performance analysis section of the paper. Overall, having each of these programs working together helps make the sentiment analysis work well in a distributed pipeline.

Performance Analysis / Results

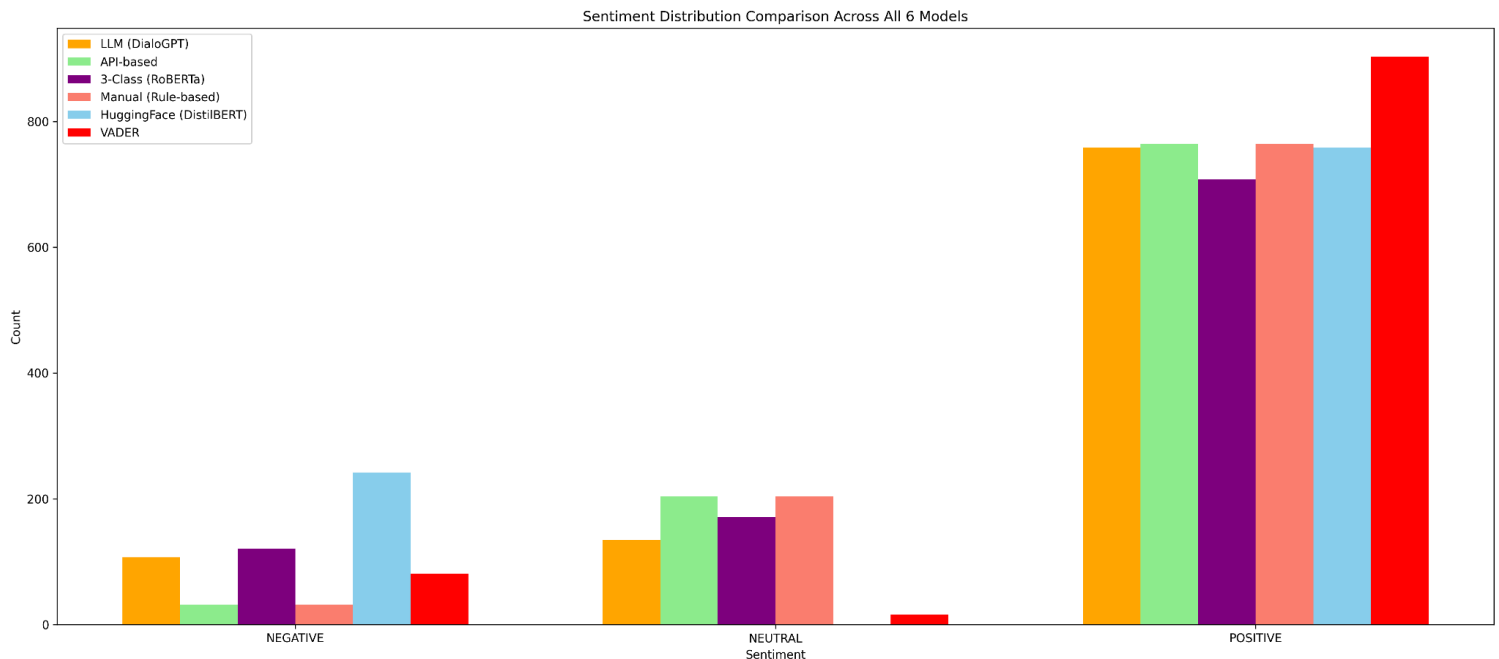
Every time this program is run there may be some discrepancies in some of the performance and results. Due to the time this program takes (around 5 minutes each run) I have decided to take the averages from three runs and analyze them here. This specific run was taken on August 5th 2025, it took twelve minutes. First let's look at some of the agreement rates between the models.



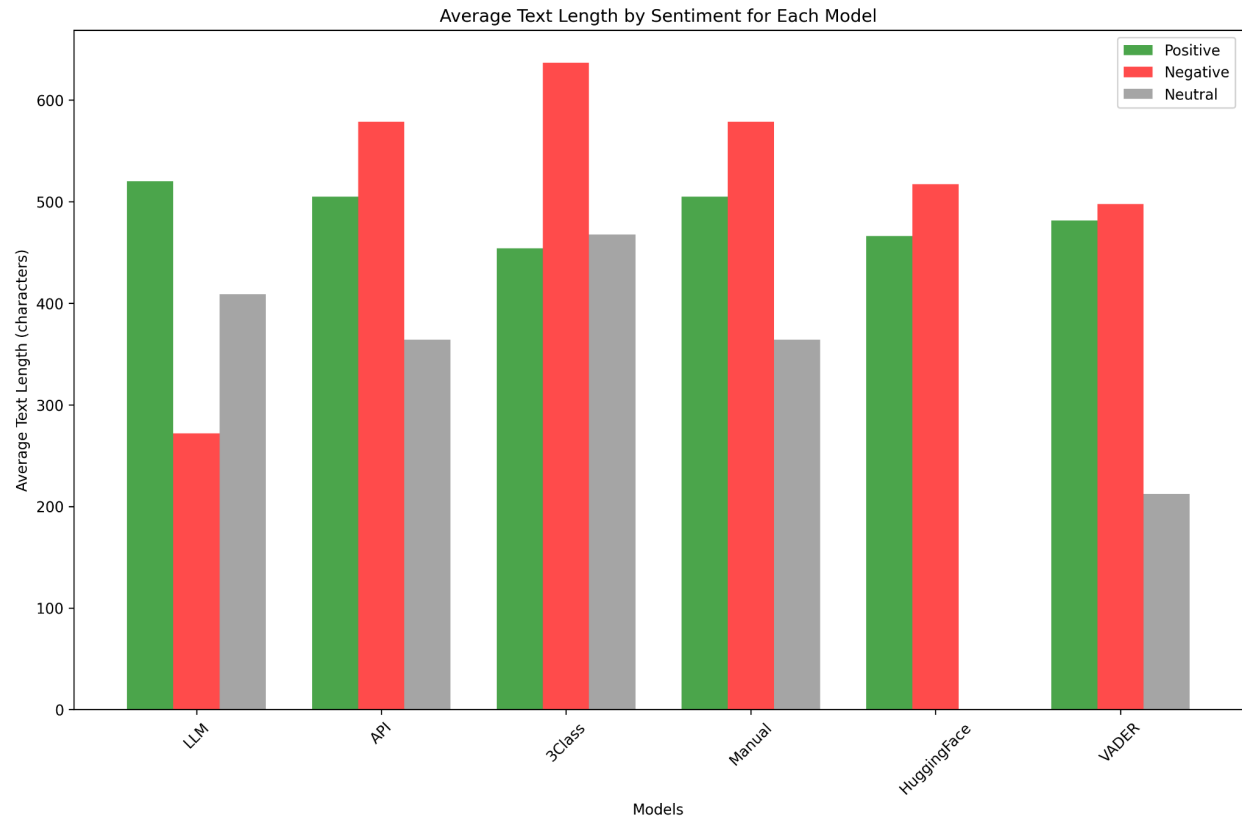
Over the six models we see that half of the time, the models unanimously agree with the analysis of each review. 31.7% of the time 4 or 5 models will agree with each other. 15.9% of the models are split, one half thinking positive, while the other half thinks negative or neutral. With only 1.1%, did two models all have different outcomes (2 positive, 2 negative, 2 neutral). Overall, the models have a solid agreement rate, with the majority having the same answer for a review. Because it was not completely one sided, it shows that there is some subjectivity in the analysis between the models. Next we have the model consistency analysis graph.



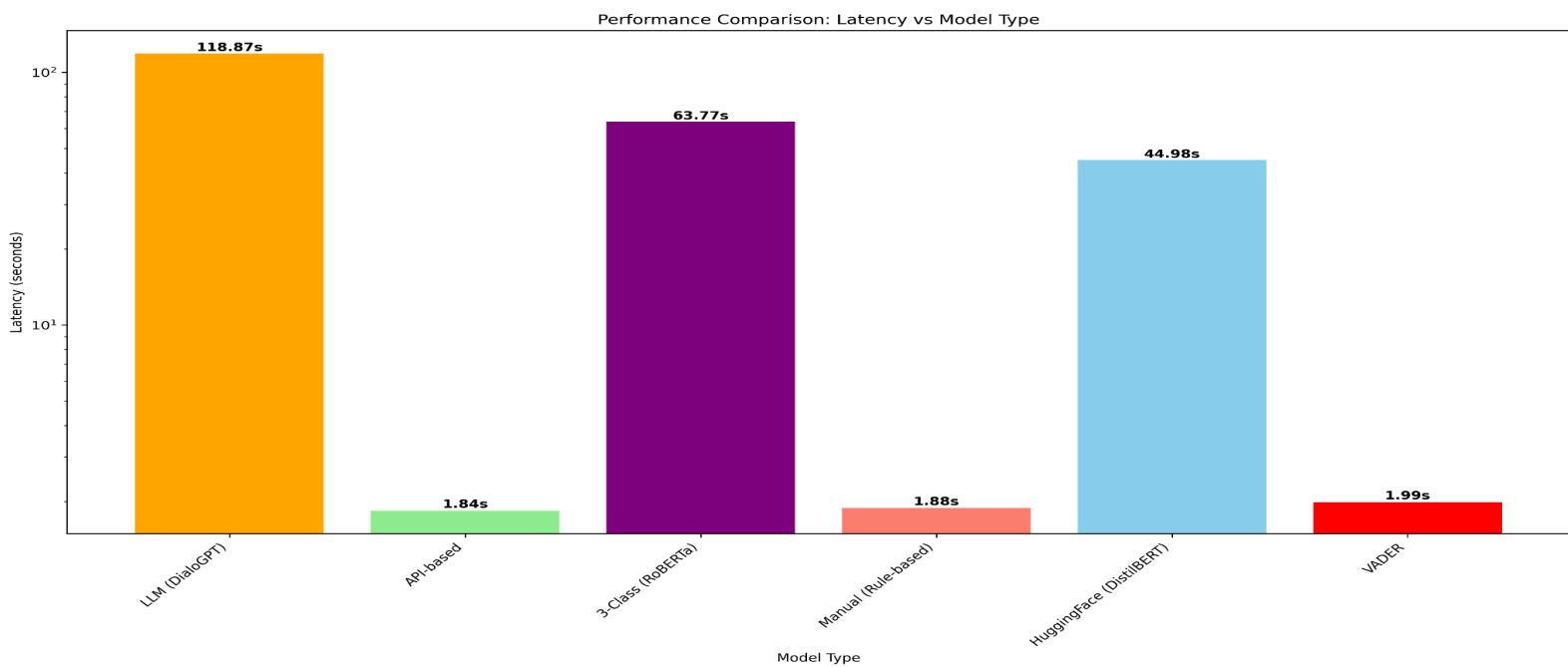
The purpose of this graph is to demonstrate which models often agree with the majority of the other models. This can suggest a more reliable analysis if the consistency is higher. Interestingly we see that our LLM model does not contain high consistency despite it being the most “sophisticated” model. Due to the algorithms being very similar, the API and manual have pretty much the same consistency. Their differences have to do more with their latency performance. We also see the VADER is one of the strongest when it comes to consistency, and both Hugging Face models (RoBERTa and DistilBERT) performing the worst. While this graph does not provide the most conclusive data, it is important to see what models agree with. The next graph we have shows how each model voted for the reviews.



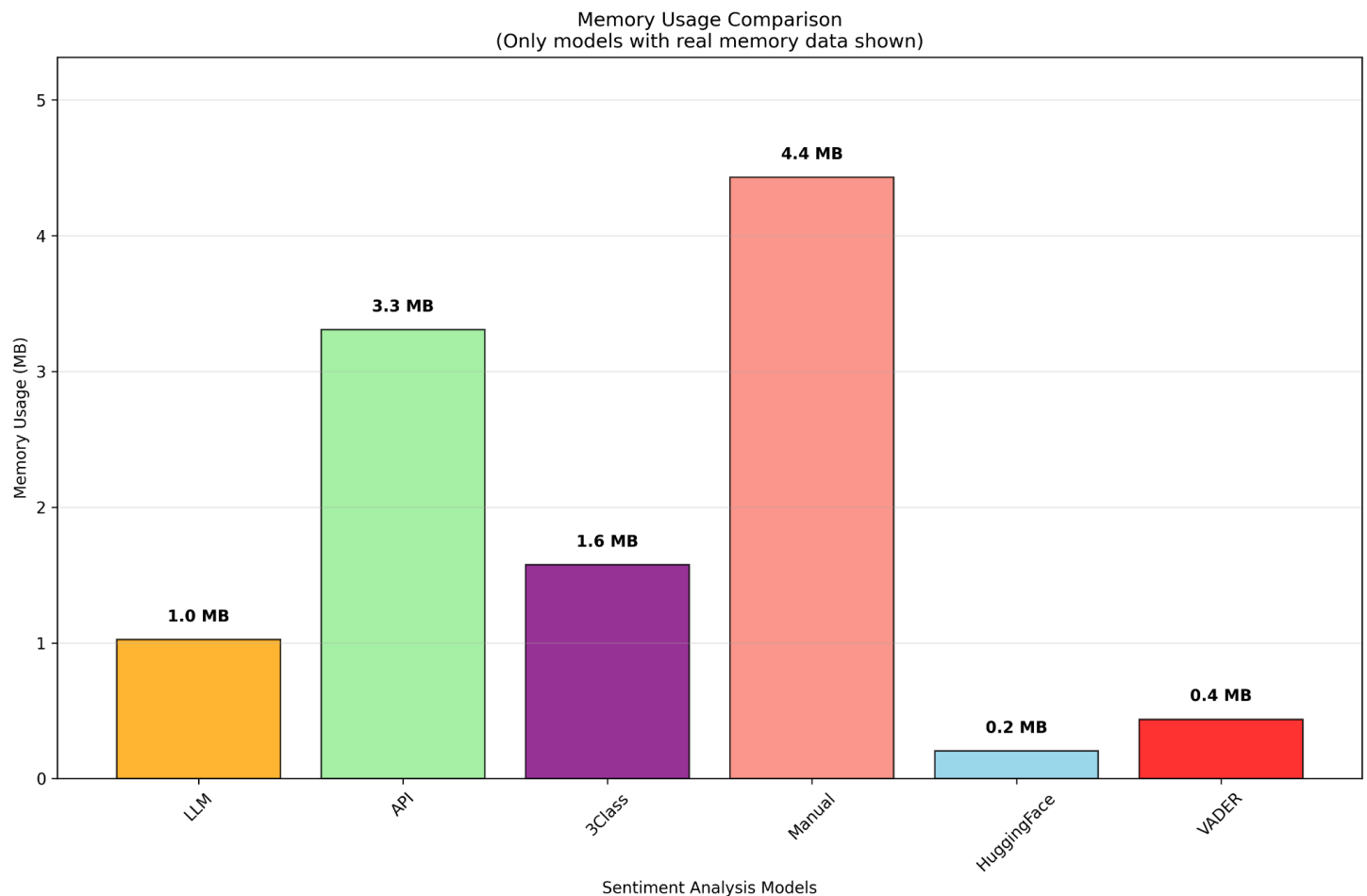
The overall majority of the time these models consider the reviews to be positive. VADER shows out the most analyzing that most of the reviews were positive, with few neutral or negatives. Besides VADER and DistilBERT (is not able to produce neutral results) most models had pretty close numbers with their neutral analysis. Negative seems to have a rollercoaster over negative reviews, with DistilBERT having the highest amount of negative reviews (from lack of neutral reviews. Overall this graph gives us some basic insight as to how the analysis voted for each review. Below we have another graph showing the average text length per sentiment by each model. While this graph does not provide a whole ton of insight, it is interesting to see how on most models, the higher the text length, the more likely the review will be negative. All but the LLM, which has shown in previous graphs it may deviate from the rest becoming more independent. Similarly most of the time, neutral reviews were lower than the positive ones as well, showing customers who had little opinion did not care to leave as detailed of a review.



Next, we will be moving on to our performance review. For this project we will mainly be looking at and analyzing the latency (time it took) and the memory usage. First let's look at the latency of each model.



We see some interesting results here. We see our rule based approach (manual and API) being the fastest. VADER is also nearly as fast due to its similar build style. RoBERTa and DistilBERT took about a minute each, and the LLM took the longest. Further analysis will be in the contributions / discussion section. For our last graph we are looking at the memory usage. This graph displays how much RAM each model used or freed during the process of running the program. In our graph, no memory was freed but mainly consumed. With the manual and API taking the most memory, and the VADER and DistilBERT taking the least. RoBERTa and the LLM taking the middle amount of memory.



Contributions / Discussion

I first want to mention that the program, results, and report were all done by myself. For this section, I will be reviewing each graph and analyzing which models are the best for this project based on the previous run and graphs above. One thing to note is that depending on certain limitations with different computers, internet access, and other factors, these results may vary. First let's look at the manual model, this model was one of the fastest models only taking an average of 1.88 seconds to process one thousand reviews. However it was by far the model that cost the most memory coming in at 4.4 MB on average of memory. This is likely due to the mixture of word counting leading to a fast result, but the algorithms needed to process create data structures to load memory into. We have a similar result to the API simulation. As mentioned before the manual and API follow a similar algorithm with some small differences in different functions. We do see a very similar output, it was slightly faster than the manual model at 1.84 seconds (negligible). This comes from the simple rule based approach. Due to the helper functions used for our API approach, we are able to optimize our processing in our memory area. However, it was still pretty high at 3.3MB on average. Going onto the Hugging Face models, we have DistilBERT which took 44.98 seconds on average, likely due to a mixture of the pre-trained model, token process, and the networking computation. Because of this we also get the most efficient memory usage at only 0.2MB, due to the optimization of the Hugging Face pipeline and efficient model loading. Next we have the 3 Class RoBERTa model. This one takes slightly longer at 63.77 seconds on average, this is due to its larger nature than the DistilBERT, it also takes a lot more memory at 1.6 MB on average as well. There is overall more computation required to perform these tasks. While it isn't the fastest or the best with memory, it is still reliable. Moving on to one of the more interesting models, we have our LLM. The

DiabloGPT-Small, is by far the most sophisticated model out of them all. However it is the slowest by a lot. Averaging a whopping 118.87 seconds per run, it is slower than the next model by nearly a minute! This is mainly due to the text generation process, tokens, and fallback mechanisms in place when the LLM produces a different output than expected. Surprisingly the model only averages 1.0 MB of data. Something to note depending on how many runs have been done in a certain amount of time, can lead to different results. Overall this one is extremely slow, but memory usage over a span of three runs is better than expected. Lastly, we have arguably our best model, VADER. VADER performed well with both latency (1.99 seconds on average) and memory usage at only 0.4 MB on average. Its speed was the third fastest, but only by one tenth of a fraction. While its memory usage was the second lowest. Due to its training and lexicon based approach, this allows for a speedy and efficient work load. While agreement rate is not necessarily the best when it comes to determining which model was the best, VADER's high performance and overall high agreement rate make it one of the best models for sentiment analysis.

Conclusion

In total we looked at six different models to perform sentiment analysis over a distributed system using the Ray framework. This project demonstrates the use of distributed systems, software design, results analysis, and even the use of LLMs and other models. Overall my findings lead me to believe that the strongest contender for this specific project was VADER, through its top tier performance for both memory and latency, as well as its higher overall agreement rate. Overall this project was both challenging yet key in helping me understand distributed computing!

References

<https://aws.amazon.com/what-is/sentiment-analysis/>

<https://www.ibm.com/think/topics/sentiment-analysis>

https://en.wikipedia.org/wiki/Sentiment_analysis

https://docs.ray.io/en/latest/ray-overview/index.html?utm_source=ray_io&utm_medium=website&utm_campaign=any_ai_workload&_gl=1*1rtx086*_gcl_au*MTkwNTQ3NjE5OC4xNzU0MDcyNzQy